

Bitcoin Witness

Proving Bitcoin without moving Bitcoin

BUIDL CTC 2026 Fall — exSat + Attestcoin + Creditcoin

A Creditcoin smart contract can now read a Bitcoin reserve balance, proven from Bitcoin itself, with no wrapping, no custodian and no bridge — and refuse to mint against reserves that are not there.

github.com/zhenek73/bitcoin-witness

1 — The problem

Every wrapped Bitcoin carries exactly one promise: **the supply on this chain never exceeds the BTC held in reserve.**

Let us be precise about the state of the art, because the lazy version of this pitch is wrong. That promise **is** monitored today. Chainlink publishes a WBTC proof-of-reserve feed, and its Secure Mint pattern already gates minting on it. **The problem is not that nobody looks.**

The problem is what the contract is actually looking at.

A reserve feed is an *assertion*. A committee of oracle nodes reads a Bitcoin node off chain, agrees on a number, signs it, and posts it. The consuming contract cannot check that number against anything — it can only trust the signers, and trust the address list those signers were pointed at.

The reserve is reported to the chain. It is never proven to it.

That was not laziness on anyone's part. **Bitcoin state has never been natively readable by a Creditcoin contract.** Other approaches to the general problem — BTC Relay, SPV light clients, threshold-signature designs like tBTC — each make their own trade-off in gas, liveness or signer trust. Bitcoin Witness takes a different route: it uses exSat's already-indexed Bitcoin state and makes one specific fact verifiable inside a Creditcoin contract.

The reserve reaches Creditcoin as a fact derived from Bitcoin's own proof-of-work-verified UTXO set, carried by an Attestcoin attestation over a real exSat block, and checked on chain by the BlockProver precompile. **The contract verifies evidence instead of trusting a reporter.**

2 — The insight

exSat already maintains a complete, native, on-chain index of the Bitcoin UTXO set — **166 million UTXOs**, kept current by real mining pools submitting real blocks with real proof-of-work.

It is one of the most under-used pieces of infrastructure in crypto: a Bitcoin state machine that smart contracts can read.

Except they can't, quite. That index lives on exSat's **native** (Antelope) layer. Solidity contracts live on exSat's **EVM** layer. exSat's own documentation says the EVM layer "*will in the near future*" be able to read it. Today it cannot — and a contract on another chain has no way to reach those tables either, not because they are secret, but because reading is not verifying.

Anyone can query them over a public EOS RPC. What no smart contract elsewhere can do is check their contents as part of its own execution.

RPC-readable is not contract-verifiable, and that gap is the whole problem.

Bitcoin Witness builds that missing link, and then carries the fact one chain further, to Creditcoin.

3 — What we built

```
Bitcoin -> exSat native index -> exSat EVM -> Attestcoin -> Creditcoin -> ReserveGuard
      (utxomng.xsat)      (event)      (attestation) (proven fact) (mint reverts)
```

1. **btcwitness11** — Antelope C++ contract, live on **EOS mainnet**. Reads a Bitcoin UTXO straight out of exSat's `utxomng.xsat` table and relays it into exSat's EVM through an inline action. Not a bridge — a table read and a function call, inside one chain.
2. **BitcoinWitnessReceiver** — live on **exSat EVM mainnet** (chain 7200). 12 real `BitcoinUtxoAttested` events, all publicly checkable.
3. **A live Attestcoin attester watching exSat**. exSat was not a registered source chain, so we ran our own Creditcoin node, registered it, bootstrapped an attester, and it attests real exSat mainnet blocks continuously.
4. **BitcoinFactVerifier** — proves the attested payload through the BlockProver precompile, then authenticates the fact *from inside the attested bytes*: right recipient, right emitter, right event signature, right relayer, successful receipt. A caller chooses only *which* proven transaction to submit, never what it says. 9 tests, 5 negative paths.
5. **ReserveGuard** + **GuardedWBTC** — the part that turns a proven fact into a safety property.

Five distinct Bitcoin UTXOs have gone the whole way and read back `proven = true` .

4 — The one line that matters

```
function mint(address to, uint256 amount) external onlyIssuer {  
    guard.checkMint(amount);    // reverts unless supply stays  
    _mint(to, amount);          // within proven Bitcoin reserves  
}
```

scripts/demo_solveny.mjs , five beats, all on chain, all passing:

#		Result
1	issuer declares its five proven Bitcoin outpoints as reserve	250 BTC counted
2	backed mint of 50 gwBTC	succeeds
3	attacker mints 250,000 gwBTC	reverts — Insolvent(250050, 250, 0)
4	<i>same attack on an unguarded token</i>	succeeds — 249,750 unbacked, no error
5	reserve proof allowed to age out	even 1 satoshi reverts

Beat 4 is the control case, deployed on purpose. It is what a mint path looks like with no reserve constraint it can verify: a valid transaction by every rule the chain knows, because the chain does not know what the reserve is.

Beat 5 is what makes this safe to rely on. Proofs have latency, so the number is always the reserve as of some minutes ago — and a deposit not yet proven is simply *not counted*. Lag makes the guard stricter, never more permissive. It fails closed by construction.

5 — What we found on the way

exSat's EVM layer serves `receiptsRoot` and `stateRoot` as 32 zero bytes in every block header — not one bad RPC node, but structurally, on every block.

Gluwa's attester recomputes those roots and compares them to the header. For an empty block the comparison happens to pass; on the **first block containing a transaction** the computed root is a real hash that can never equal a permanent zero — so the attester rejects the block as a "possible reorg", retries forever, and never advances again.

exSat cannot be attested by stock Attestcoin, and the error points at the wrong cause entirely.

We proved which parts of the chain are and are not sound: `transactionsRoot` is genuine and canonical (recomputed and matched byte for byte — that is the field inclusion proofs actually use), the header hash is self-consistent, and in Gluwa's own source the receipts root enters **no attestation and no proof**.

Both defects filed upstream, against Gluwa's repository:

- **gluwa/creditcoin3#1355** — the attester cannot follow any chain without a receipts trie, and reports it as a reorg that never happened.
- **gluwa/creditcoin3#1356** — merged-ready patch: `register_bls` logs "already registered" for an account that is not registered at all, sending operators looking in exactly the wrong place. **CI green**.

6 — What we are honest about

Most hackathon projects say "trustless" and stop. The actual trust model:

- We do **not** run SPV. We inherit exSat's Bitcoin consensus — miners' PoW plus exSat's validator set — the same assumption exSat's own users already make.
- Below that sits **EOS mainnet finality**; above it, **Attestcoin's quorum**.
- exSat's Bitcoin index is currently **frozen at Bitcoin height 959,115**. We found this ourselves and did not paper over it: every proven fact carries its `sourceHeight` on chain.

How this really compares to an oracle reserve feed. The difference is real, but it is not "trust versus no trust" — both designs end in a quorum. The difference is what the quorum is asked to do. An oracle committee is asked to *assert a balance it read off chain*, and the contract has no way to disagree. Attestcoin's quorum is asked to attest a **block header**, and the contract then verifies for itself that the fact was inside that block and that the block belongs to the chain. Corrupting the first produces a wrong number nothing on chain can detect. Corrupting the second means forging exSat's history — and with it the Bitcoin data exSat's validators endorsed under proof of work.

Three limits of the guard we would rather state than have someone find:

- **It stops inflation, not theft.** Draining already-backed tokens leaves the invariant intact. "You cannot print what is not there" — never "funds cannot be stolen".
- **The issuer declares its own reserve outpoints.** A lying issuer is still a lying issuer — but the lie becomes a single public, permanent, checkable statement, and outpoints are globally unique, so double-claiming is detectable by anyone.
- **A proven UTXO can be spent the next block.** That is what the freshness window is for, and why it fails closed.

Proving a UTXO exists is **not** a liquidatable collateral claim, and we do not pitch it as one. Seizing Bitcoin on default needs a lock on the Bitcoin side. We are the proof layer such a lock would report through — not the lock.

7 — Why this matters, and status

Once a Creditcoin contract can read a Bitcoin fact:

- **Solvency as a transaction precondition.** Any issuer of a Bitcoin-backed token can adopt `ReserveGuard` as-is — and the reserve it gates on is verified, not reported.
- **Proof of reserves that is actually a proof.** Not a number a committee signed after reading a node, but one the reading contract checks for itself, at outpoint granularity, in the same transaction it acts on.
- **Proof of payment.** A payment is an event, not a balance, so a point-in-time proof is exactly the right instrument. "This borrower sent 0.4 BTC at height N" — recorded by Creditcoin as repayment. That is Creditcoin's own product.
- **Creditworthiness without seizure** — the underwriting signal for undercollateralized credit, which is Creditcoin's thesis.

Piece	State
Native relay contract	deployed, EOS mainnet , 12 real relays
EVM receiver	deployed, exSat EVM mainnet , 12 events
Creditcoin devnet + exSat registered	working , rebuildable in one command
Attestor against exSat mainnet	working — attesting live blocks
Creditcoin verifier	deployed , 9 tests incl. 5 negative paths
ReserveGuard + GuardedWBTC + control	deployed , 5-beat solvency demo passing
Live end-to-end run	done — 5 UTXOs proven , transcripts in repo

Everything above was verified against running software and live chain queries, not inferred from source.